Silvia Sisinni

silvia.sisinni@polito.it

Module F - Trusted Computing and Remote Attestation

L1 - Introduction to Trusted Computing

Politecnico di Torino

Department of Control and Computer Engineering

TORSEC

Computer and Network Security Group

License & Disclaimer

License Information

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.
- This material is used and adapted within the course “Cybersecurity and National Defense”.

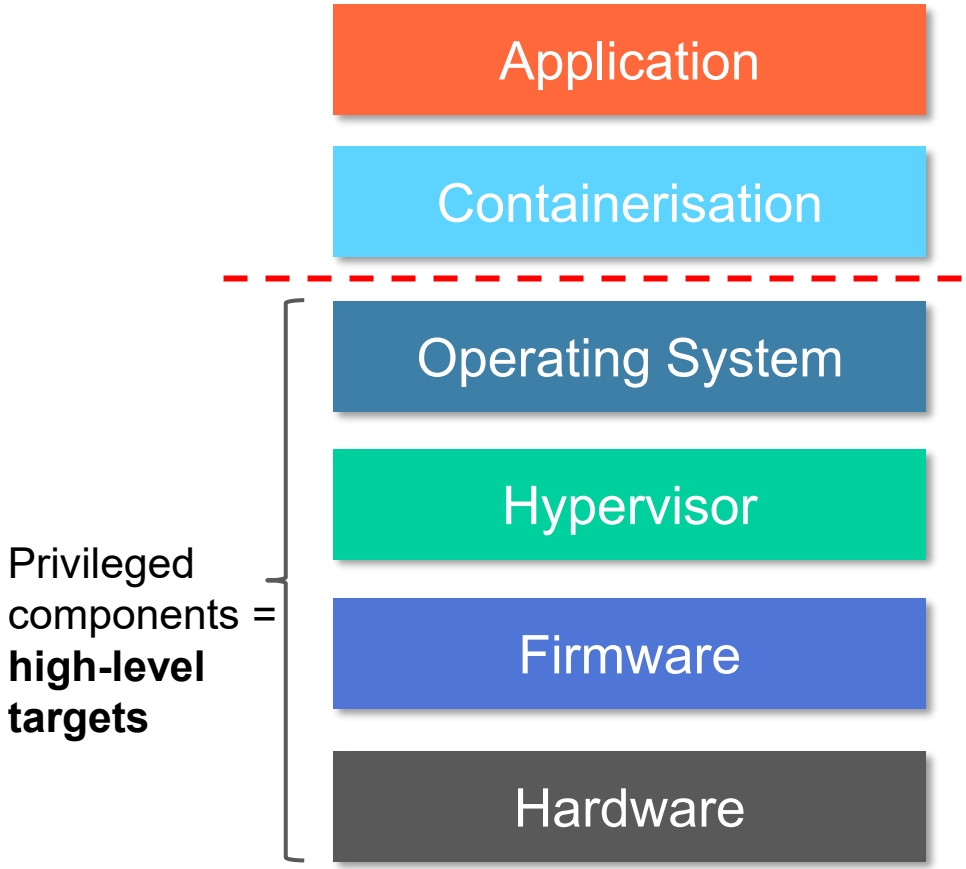
Why trust became a system-level problem

- Modern platforms process **large volumes of sensitive data**
- Data is increasingly shared across **devices, clouds, and organisations**
- Security failures affect **confidentiality, integrity, availability, and accountability**
- Classical security mechanisms have historically focused on two important states of data:
  - **Data at rest** (stored on a disk, in a database, or in a file system) ⟹ full-disk encryption, file-based encryption, database encryption, and access-control policies
  - **Data in transit** (data moving over a network) ⟹ TLS, IPsec, …
- The hard problem is protecting data **during computation**

? Can we trust the platform that is processing the data?
Has this platform been compromised by an attacker?
How is it possible to verify its real status?

Layered security helps, but it expands the trust assumptions

- Modern systems use many layers of protection (**Defence in Depth**):
  - TLS for data in transit
  - encryption for stored data
  - Operative Systems isolate processes
  - Sandboxing (e.g. containers) better isolates processes
  - access control mechanisms (SELinux, AppArmor)
  - Virtual Machines
- If one control fails, another may still limit the damage ..
- But most layers still depend on **privileged software** behaving correctly.



The operating system is powerful ⟹ it is dangerous when compromised

- The operating system has extensive control over:
  - user separation
  - process scheduling and isolation
  - virtual memory and page tables
  - file systems and storage
  - network interfaces
  - device drivers and I/O
- A kernel-level attacker may observe memory of all user-space processes, modify page tables, or interfere with system calls, read-write files, control network interfaces, communicate with peripheral devices, …
- Large operating systems have large attack surfaces (millions lines of code)
- Modern attacks exploit hardware- and microarchitectural behaviour (e.g., cache, speculative execution, optimization mechanisms), not only softw bugs

The trust problem

- **Secure ≠ Trusted**
- A **secure system** enforces a defined security policy under a defined threat model
  - strong claims of security require strong assurance
  - this may involve formal verification, systematic testing, or certification.
- In practice, modern systems are too large to be fully verified …
- A **trusted system** is expected to behave in a predictable way
  - requires assumptions and evidence
  - **trust does not imply absence of vulnerabilities.**
- Example: we expect a bank to behave like a bank by offering financial services, and a thief to behave like a thief by stealing
  ⇒ trustworthy behavior ≠ correct or secure behavior

Two complementary responses: verify the platform or isolate the computation

- **Trusted Computing (TC):** verify whether a platform is in an expected state.
  - Detects unexpected software components and configuration
  - Main primitive: **Remote Attestation**
  - Question: *"Is this platform in an expected state?"*

- **Trusted Execution Environments (TEEs):** protect sensitive code and data from the rest of the platform.
  - Protects code and data **in use**
  - Main primitive: protected execution
  - Question: *"Can this computation remain protected from privileged code (OS and hypervisor)?"*

Trusted Computing: making platform state verifiable

Question: *"Is this platform in an expected state?"*

Trusted Computing provides mechanisms to:

1. **Measure** software and configuration state
   - a measurement is typically a cryptographic hash of a software component or configuration
   - Example: the firmware may measure the bootloader, the bootloader may measure the kernel, and the kernel may measure additional components.
2. **Protect** measurements in hardware
   - ensure that compromised software cannot simply rewrite the history of measurements to make the system appear clean after loading malicious code
3. **Report** measurements to a verifier
   - produce a signed statement about system integrity measurements
4. **Decide** on the platform state:
   - bind secrets and sensitive data to an approved platform state

Confidential Computing: protecting data in use

Question: *"Can this computation remain protected from privileged code (OS and hypervisor)?"*

- Encryption protects data **at rest**
- TLS protects data **in transit**
- The hard problem is protecting data **during computation**
- **Confidential Computing** aims to provide mechanisms for secure computation even when the surrounding platform, service provider, or other parties are not fully trusted.
- Confidential Computing encompasses three main families:
    1. **Trusted Execution Environments (TEEs)**
    2. **Fully Homomorphic Encryption (FHE)**
    3. **Secure Multi-Party Computation (MPC)**

TEEs: reducing trust in the normal execution environment

- A TEE protects selected code and data from the normal execution environment, also called Rich Execution Environment (REE).
  - It reduces dependence on the normal OS or hypervisor.
- Typical security goals:
  - **confidentiality** of sensitive data in use
  - **integrity** of code and data in use
  - **isolation** from the normal OS/hypervisor
  - **attestation** in some TEE designs
  - **trusted I/O** in some implementations

The TEE objective is not to secure the whole platform,
but to protect a small execution environment under a specific threat model

FHE:computation without decrypting data

- **Fully Homomorphic Encryption (FHE)** allows computation directly over encrypted data
- A server can evaluate a function without seeing plaintext inputs
- Only the data owner and key holder can decrypt the final result
- Example idea:

$$\text{Enc}(3) + \text{Enc}(5) \rightarrow \text{Enc}(8)$$

- **Trust model:** trust the cryptographic scheme, not the computing platform
- **Main cost:** very high computation overhead compared with native execution, not practical for workloads requiring low latency, high throughput, or limited power consumption

SMPC: collaboration without revealing private inputs

- **Secure Multi-Party Computation (SMPC/MPC)** lets several parties to compute a result together by using cryptographic protocols, without revealing their private inputs to the others.
- Each party keeps its own input private
- No single party needs to receive all raw data
- Examples: several organisations compute a shared statistic, detect fraud, or perform collaborative analysis without sharing datasets
  - with ordinary computation, this would require one party to reveal data to another party, or all parties to send their data to a central trusted entity
- Useful when parties do not fully trust each other
- **Trust model:** trust the protocol, not a central data holder or the participants
- **Main cost:** several rounds of communication, protocol **complexity**, and higher computational and network overhead than ordinary computation

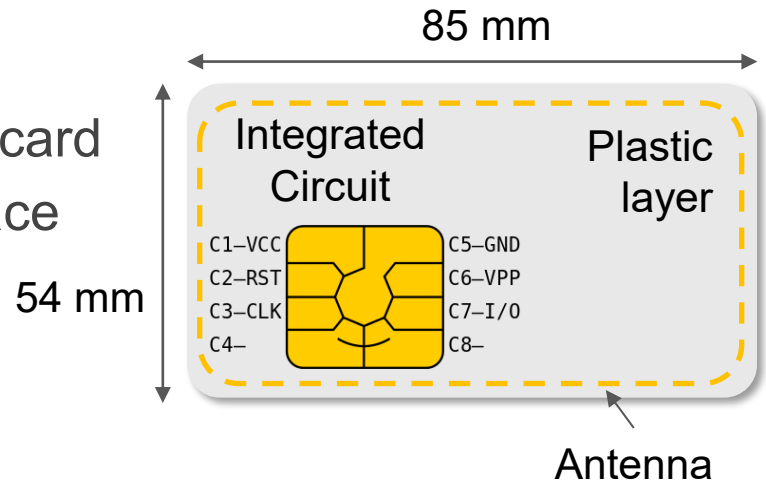
Comparing Confidential Computing approaches

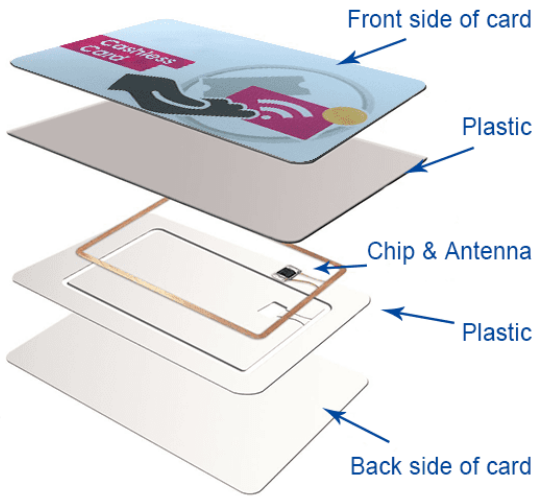
Approach	How it protects computation	Strength	Limitations
TEE	Isolate execution using hardware support	High-performance protected computation	Different TEEs provide different security guarantees; portability issues of trusted applications between TEEs
FHE	Compute directly on encrypted data	Strong cryptographic privacy	Computation overhead
SMPC	Split computation among parties	Joint computation without sharing raw inputs	Protocol complexity; difficult to deploy at scale

Isolated Hardware Execution Platforms

Smart Cards: the classical isolated security platform

- Embedded **microprocessor + memory** inside a card
- Standardised physical and communication interface
  - contact: **ISO/IEC 7816**
  - contactless: **ISO/IEC 14443 / NFC**
- Contactless cards can be powered by the electromagnetic field generated by the reader, so the card does not need its own battery
- Used for: payments and transport ticketing, SIM authentication, electronic identity documents, physical access control
- Stores **keys, identifiers, and application data** internally
- Performs cryptographic operations **inside the card**
- Exposes only controlled command interface to the outside world, not raw secrets



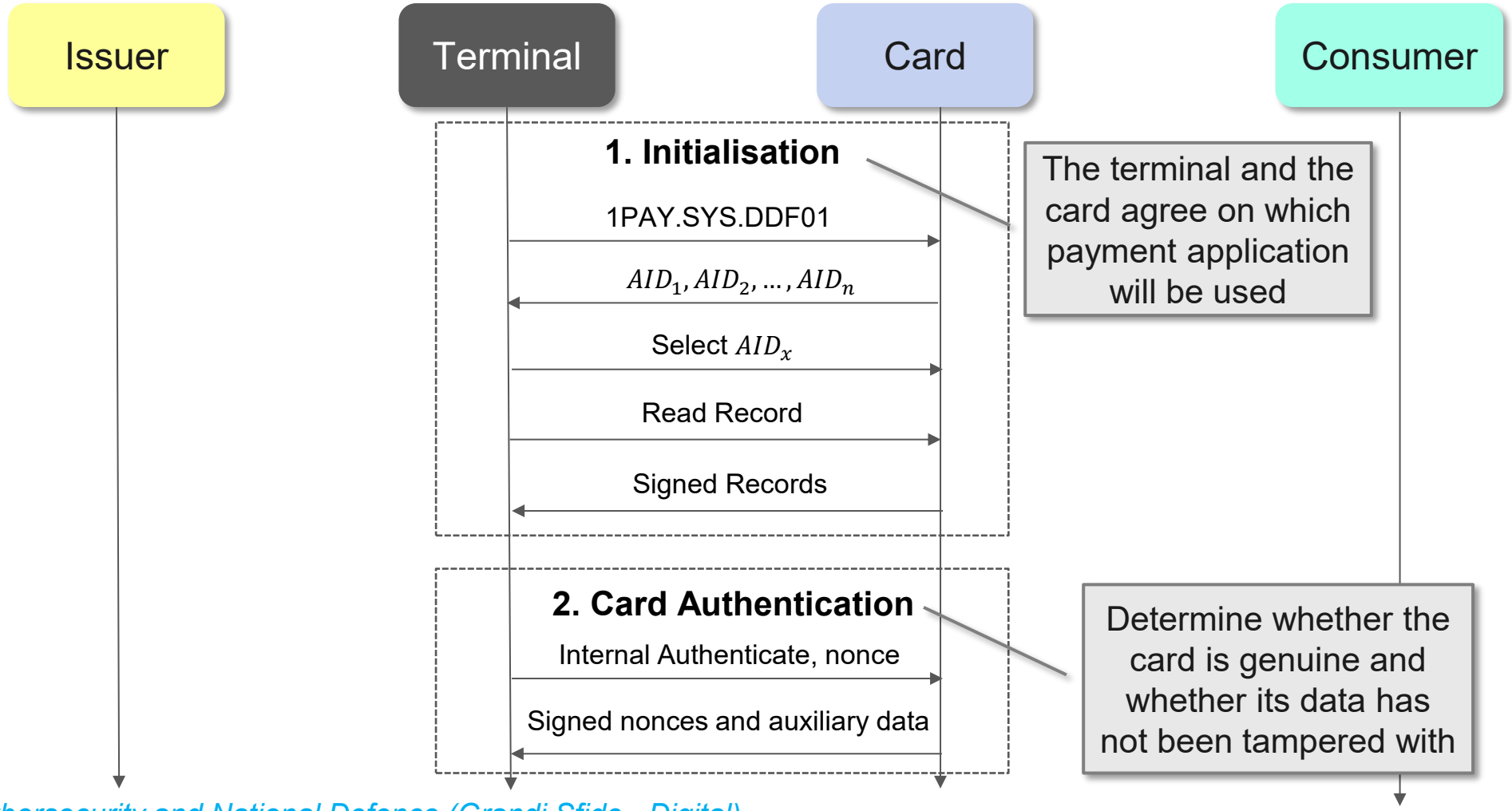


EMV: smart cards for payment security

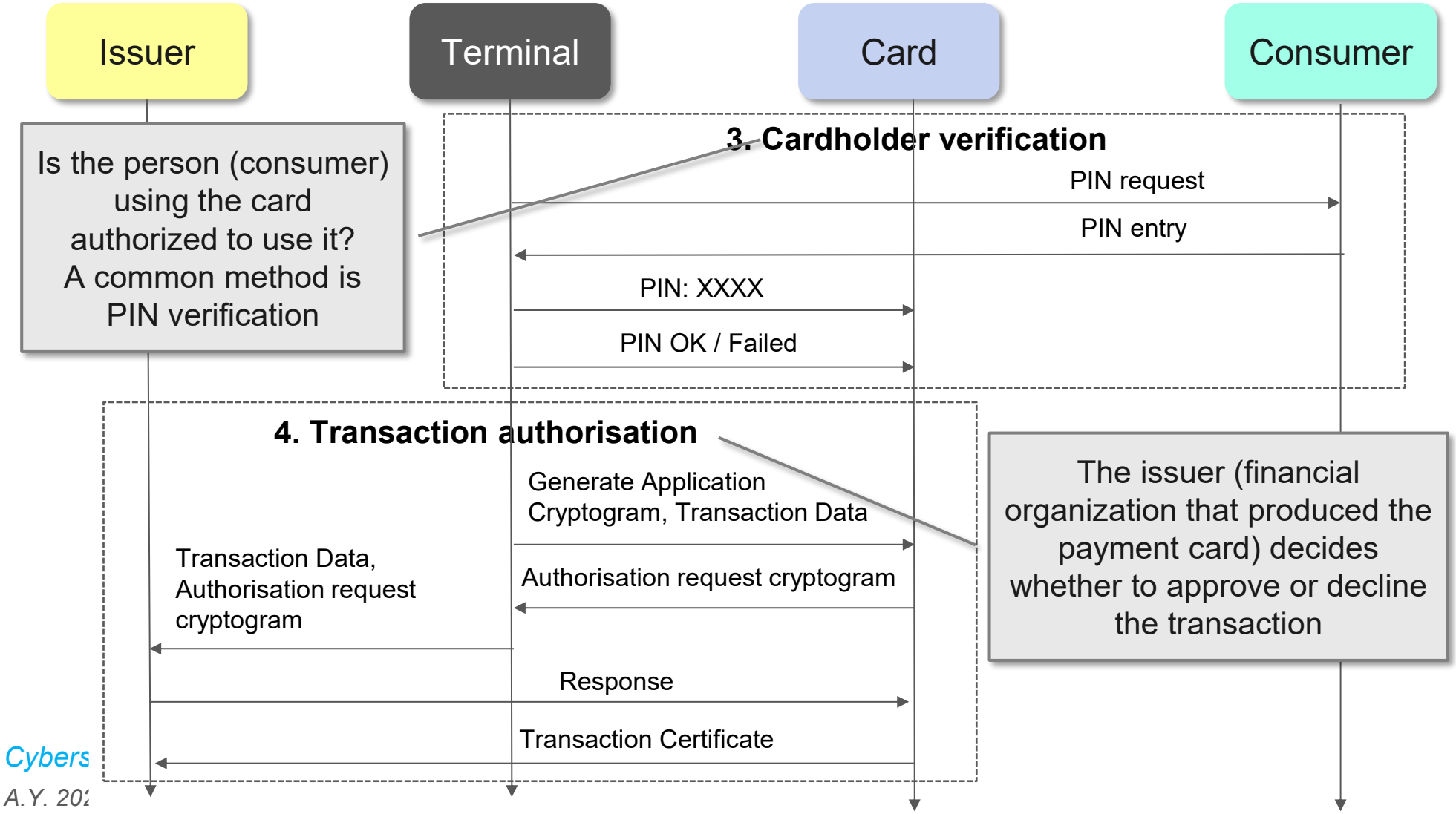
- **Europay, Mastercard, and Visa (EMV)** defines how chip-based payment cards interact with terminals
- Motivation: replace magnetic-stripe cards with stronger cryptographic authentication
- In an EMV transaction, the smart card participates actively in the payment protocol: it stores cryptographic keys, processes commands from the terminal, verifies data, and generates cryptographically protected messages that prove its participation in the transaction

- A payment card acts as a **secure execution environment** for: card authentication, cardholder verification, transaction cryptogram generation, protection of payment keys
- The **terminal**: selects a payment app, sends challenge, sends transaction data;
- The **card**: authenticates, checks PIN, generates cryptograms
- The **issuer**: verifies transaction and authorises or rejects it

Simplified EMV transaction flow (I)



Simplified EMV transaction flow (II)



What EMV teaches us about hardware security

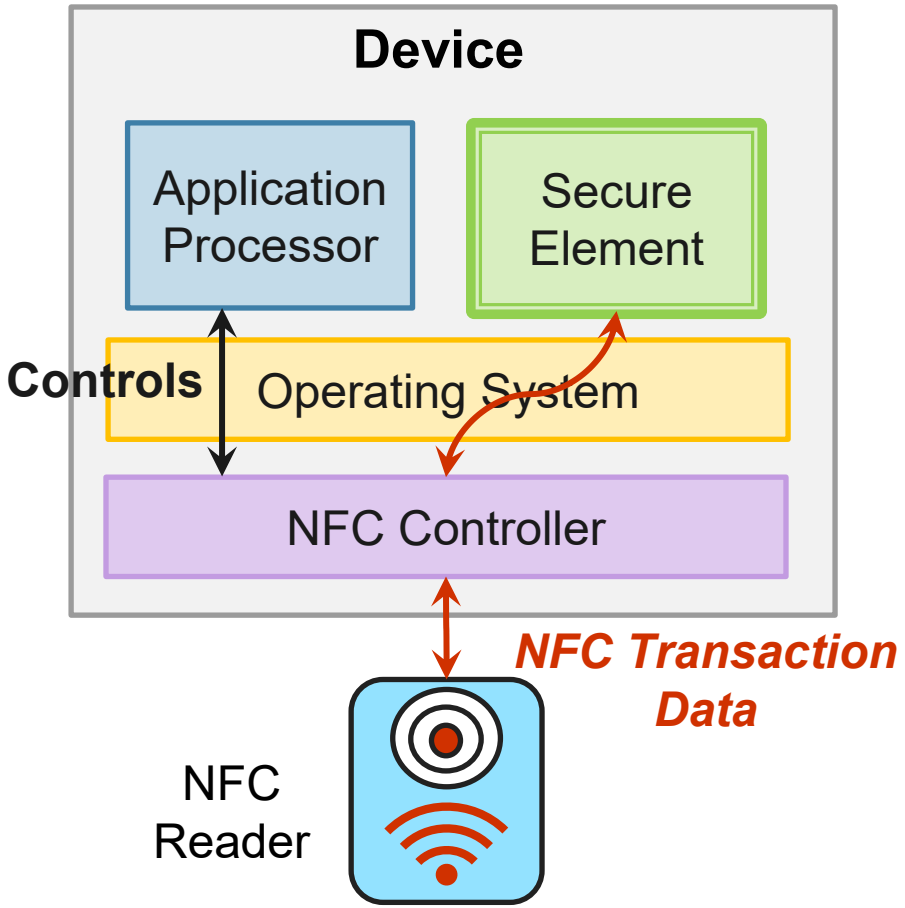
1. Keep long-term secrets inside isolated hardware
2. Expose operations only though a controlled command interface, not keys
3. Use challenges/nonces to prevent replay attacks
4. Bind cryptographic responses to transaction data
5. Standardise protocols for global interoperability
6. Certify platforms when high assurance is required
    - payment cards operate in adversarial environments and protect high-value assets
    - for this reason, smart cards and secure elements are usually evaluated under formal certification schemes such as Common Criteria or FIPS-related frameworks

Secure elements: smart-card technology embedded into devices

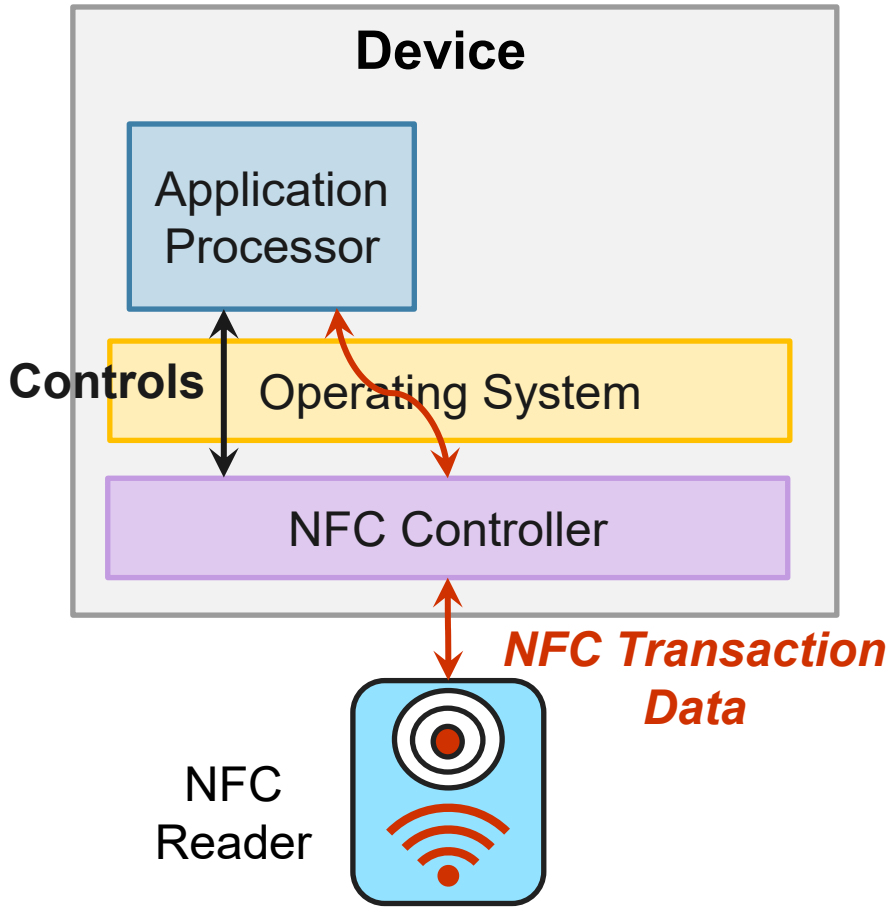
- **Secure Elements (SEs)** generalise smart-card technology
- They generalise the smart-card model beyond plastic cards
- Main form factors:
  - physical smart card
  - SIM / UICC
  - embedded Secure Element (eSE)
  - secure microSD
- Used in smartphones, embedded systems and mobile services
- Protect: subscriber identities in mobile networks, payment credentials for contactless transactions, authentication keys, security-sensitive applications
- A SE is a separate hardware component, with its own processor / memory: it guarantees stronger physical attack resistance than ordinary software isolation

SE vs Host-based Card Emulation (HCE)

Secure Element-based NFC



HCE-based NFC



GlobalPlatform SE: who controls what inside the Secure Element?

Problem: several stakeholders may need the same Secure Element for storing sensitive applications and credentials

Example: one stakeholder may be the device manufacturer, another may be a mobile network operator, another may be a payment provider, and another may be an electronic identity provider.

GlobalPlatform addresses this using the concept of Security Domains:

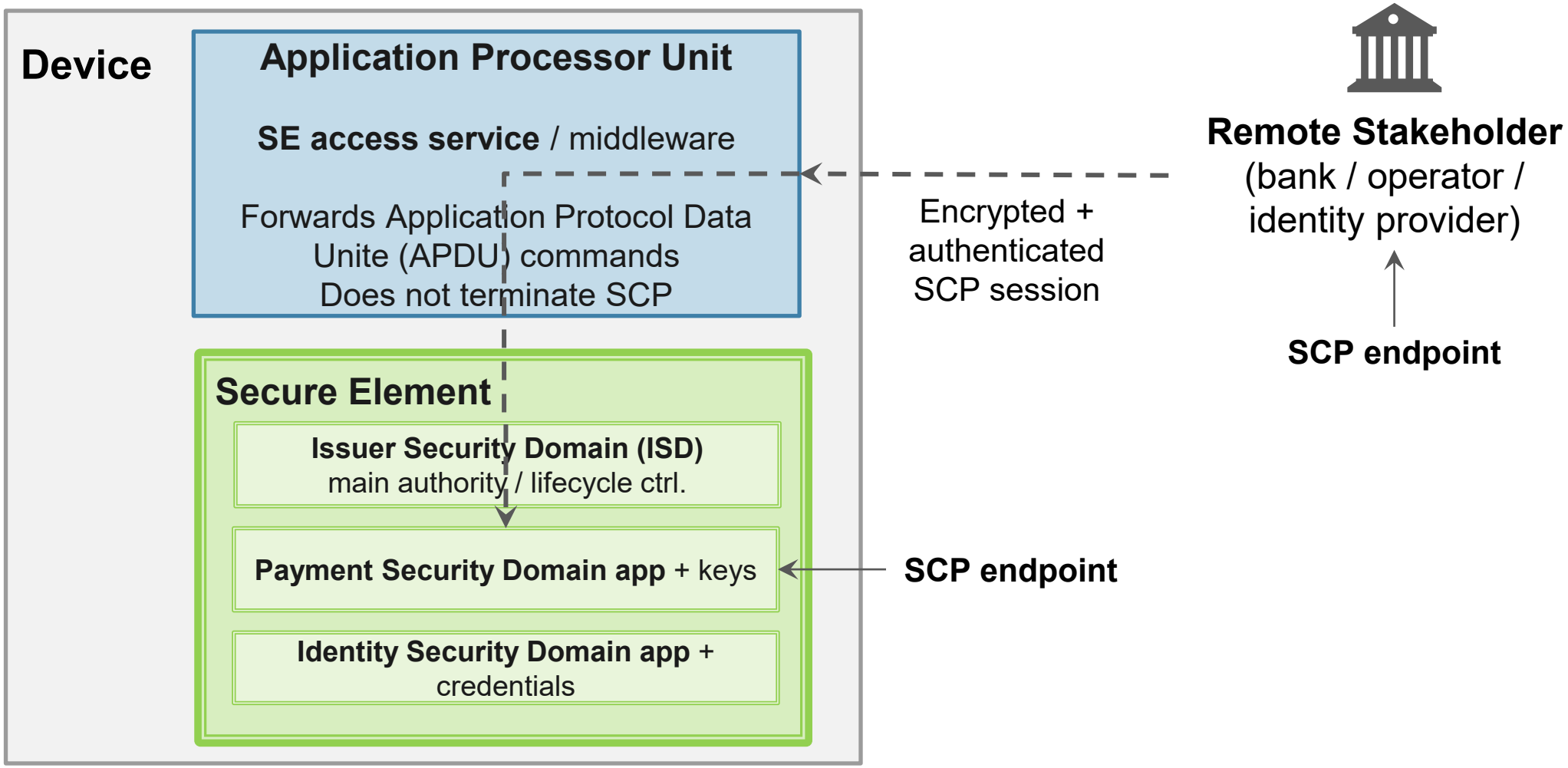
1. **Issuer Security Domain (ISD)**: issuer of the SE, main authority over the SE
   - control high-level management operations and can authorise the creation or management of other domains
2. **Security Domains (SDs)**: separated areas for service providers
   - for example: a payment provider may manage a payment application and its keys, while a mobile operator may manage SIM-related credentials
   - each SD manages the applications / secrets that belong to a given stakeholder

GlobalPlatform SE: who controls what inside the Secure Element?

3. **Secure Channel Protocols (SCPs)**: protected remote management
   - provide confidentiality, integrity, authentication, and data-origin protection for management operations
   - if a provider installs an applet in the SE, or update a key, those commands must be protected
4. **Lifecycle management:** install, personalise, update, delete, lock

Result: applications and credentials can coexist without exposing each other's secrets.

GlobalPlatform SE



Roots of Trust and Security Platform Certification

Roots of Trust: the starting point of platform trust

A **Root of Trust (RoT)** is a component (hardware, firmware, or software) that is trusted to perform one or more security-critical functions correctly.

Typical properties:

- it is **implicitly trusted**
- it has a **small and protected implementation**
- its misbehaviour is impossible to detect during runtime, when it is deployed on a platform
- the correct implementation of its security functionalities are typically certified by certification bodies
- other security mechanisms rely on it as a foundation

A **Root of Trust** is where the trust chain begins.

What can a Root of Trust do?

A Root of Trust may provide different security services:

Function	Purpose
Root of Trust for Measurement (RTM)	Compute evidence (e.g., cryptographic hashes) about software or configuration state
Root of Trust for Storage (RTS)	Protect keys, secrets, or measurements from unauthorized access or modification
Root of Trust for Reporting (RTR)	Produce authenticated evidence about the platform or component state for a verifier
Root of Trust for Detection (RTD)	Check whether code or data is authorized, before allowing execution
Root of Trust for Update (RTU)	Protect firmware or security-critical updates

Hardware Security Modules as RoTs

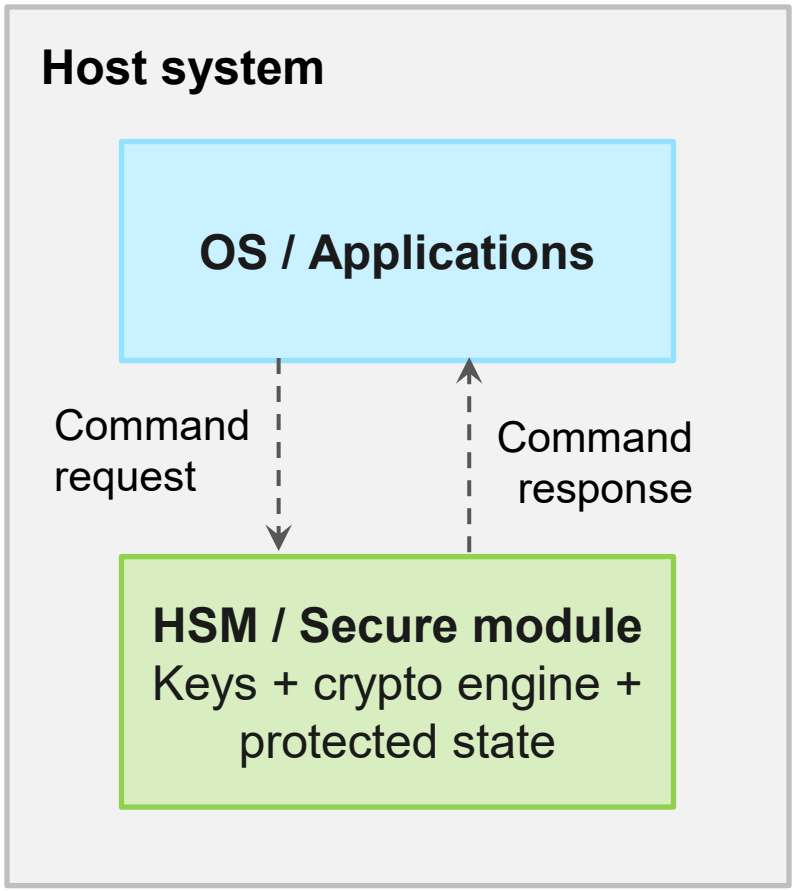
A Hardware Security Module (HSM) can act as a **Root of Trust** when it provides protected, security-critical functions that the rest of the system relies on.

Typical RoT services in HSM modules:

- **Key generation and storage** inside protected hardware, **without exporting the private key material** to the host operating system
- **Cryptographic operations**
- **Device or user authentication**
- **Integrity-protected internal state**
- **Small command interface** exposed to the host

The host can request operations, but the HSM protects the trust anchor.

Host system – HSM communication



What the host requests:

- sign(data)
- decrypt(blob)
- verify(PIN)
- read status
- ….

What the host does not see:

- private keys
- master secrets
- internal state

Certifying a security platform

High-assurance platforms are often evaluated against explicit requirements, threat models, and certification schemes.

Certification provides structured assurance that a platform was evaluated against a defined set of security requirements.

Why it matters for HSMs / SEs / smart cards:

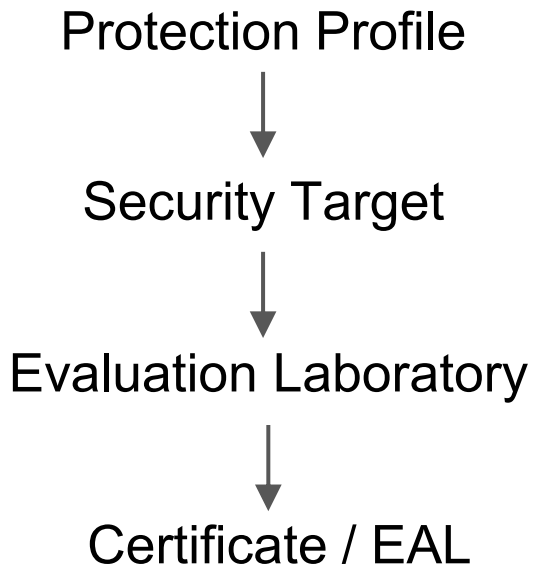
- security claims must be tested against a ***defined threat model***
- customers need evidence beyond vendor promises
- certification supports high-assurance domains:
  - payments
  - identity documents
  - government systems
  - telecommunications
- certification helps compare products with similar security goals

Important: certification is not a proof of absolute security. It certifies correct implementations of the threat protection mechanisms considered in the threat model

Common Criteria: evaluating security claims

Common Criteria is an international framework for evaluating IT security products, formalised as ISO/IEC 15408.

Term	Meaning
Target of Evaluation (TOE)	Product/system being evaluated
Protection Profile (PP)	Generic security requirements for a class of products
Security Target	Implementation-specific security requirements defined in a PP
Security Functional Requirements (SFRs)	What security functions must do
Security Assurance Requirements (SARs)	How deeply and rigorously SFRs are evaluated
Evaluation Assurance Level (EAL)	Evaluation depth and rigour



EALs: how much assurance?

Level	Meaning
EAL 1	**Functionally Tested:** This is the lowest Evaluation Assurance Level (EAL). It provides basic confidence that the TOE works as described in its documentation and can withstand basic, predefined threats.
EAL 2	**Structurally Tested:** This level gives low-to-moderate assurance that the TOE follows reasonable security practices, even when full development history is unavailable. It is often used for older or legacy systems where evidence such as test results and design documentation may be limited.
EAL 3	**Methodically Tested and Checked:** This level provides moderate assurance that the TOE has been carefully examined and tested, without requiring the higher costs associated with extensive security engineering.
EAL 4	**Methodically Designed, Tested, and Reviewed:** This level provides moderate-to-high assurance that security has been built into the TOE through systematic design, testing, and review, requiring a significant engineering effort from the organisation.
EAL 5	**Semi-formally Designed and Tested:** This level provides high assurance through a rigorous and well-planned development process, although it does not necessarily require advanced specialist security engineering techniques.
EAL 6	**Semi-formally Verified Design and Tested:** This level provides additional assurance for TOEs used in high-risk environments, where strong protection of valuable assets is required and significant security-related costs are justified.
EAL 7	**Formally Verified Design and Tested:** This is the highest EAL, used for TOEs operating in extremely high-risk environments where very high-value or critical assets must be protected.

FIPS: certification for cryptographic modules

FIPS standards are developed by the US National Institute of Standards and Technology (**NIST)** and are widely used for cryptographic security.

FIPS 140-3 focuses on cryptographic modules (hardware modules, software libraries, firmware, or hybrid implementations that performs cryptographic operations):

- approved cryptographic algorithms
- key generation and key management
- physical security requirements
- authentication and access control
- testing and validation

FIPS Security Levels

Level	Meaning
Level 1	This is the lowest security level, intended for environments where stronger security would be too costly or unnecessary. Cryptographic modules at this level must support at least one approved cryptographic algorithm and provide basic protections, such as role-based authentication and simple key generation and management.
Level 2	This level is intended for environments where sensitive information needs protection, but the risk is not critical. Cryptographic modules at this level must support at least one approved cryptographic algorithm and include stronger security measures, such as physical protection controls and improved key generation and management.
Level 3	This level is intended for environments where protecting sensitive information is especially important. Cryptographic modules at this level must support at least one approved cryptographic algorithm and include stronger safeguards, such as tamper-evident packaging and logical access controls.
Level 4	This is the highest security level, intended for environments where extremely sensitive information must be protected. Cryptographic modules at this level must support at least one approved cryptographic algorithm and include the strongest safeguards, such as physical security controls, logical access controls, and tamper-resistant packaging.

Trusted Computing Foundations: TPM and Platform State

How can a computer provide reliable evidence about the software and configuration state in which it started?

The Trusted Computing problem

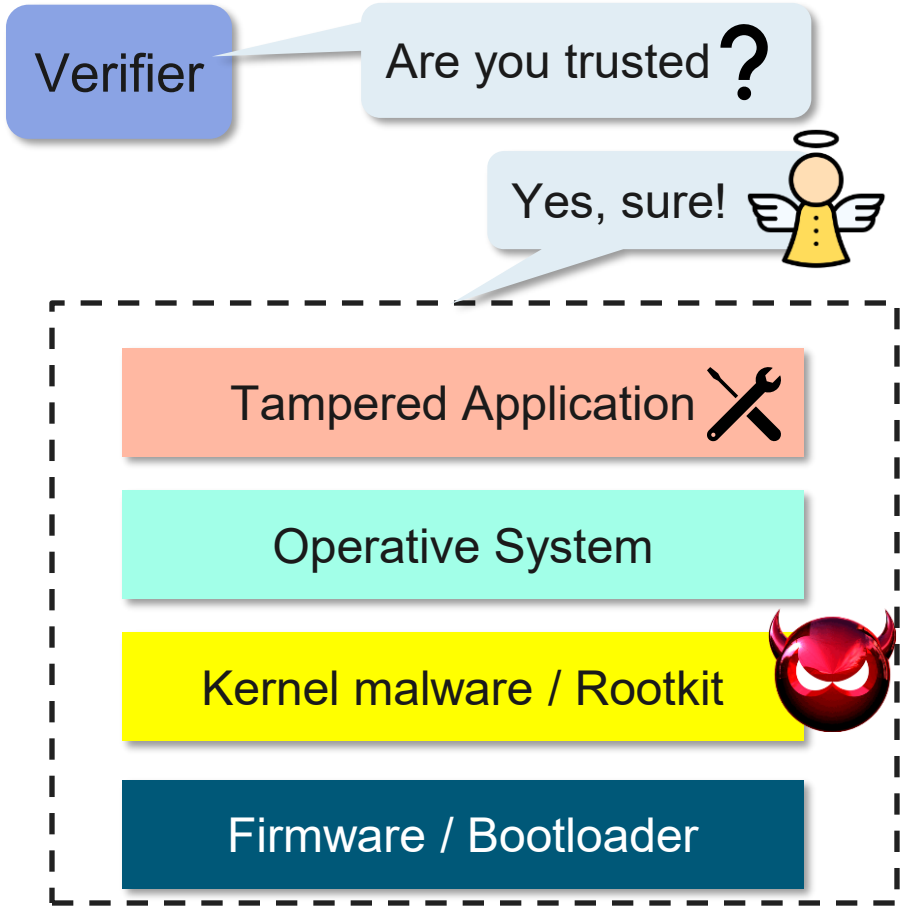
Can I know whether this platform is in the expected state ?

What is running?	What was loaded?	Has any config changed?
Runtime Firmware	Bootloader	Configuration
Kernel	Expected Drivers	Boot Policy
User Applications	Services	Security Settings

Software can report “I am fine”
even when it has already been compromised.

Why software-only verification is not enough

Compromised platform



Attacker capabilities

A privileged attacker may:

- intercept verification requests
- forge status reports
- hide modified files or processes
- alter logs before they are read
- disable security tools
- replay old “healthy” evidence

⟹ **If the verifier depends only on compromised software, the evidence is not reliable.**

The TPM: the TCG answer to platform trust

1980s
Trusted systems and the Trusted Computing Base (TCB) idea

1990s
PCs + Internet expose software-only security limits

1999
Trusted Computing Platform Alliance (TCPA)

2003
Trusted Computing Group (TCG)

2004
Trusted Platform Module (TPM) 1.2

2014
Trusted Platform Module (TPM) 2.0

The **Trusted Platform Module (TPM)** was proposed as a small, low-cost security component, attached to the motherboard to:

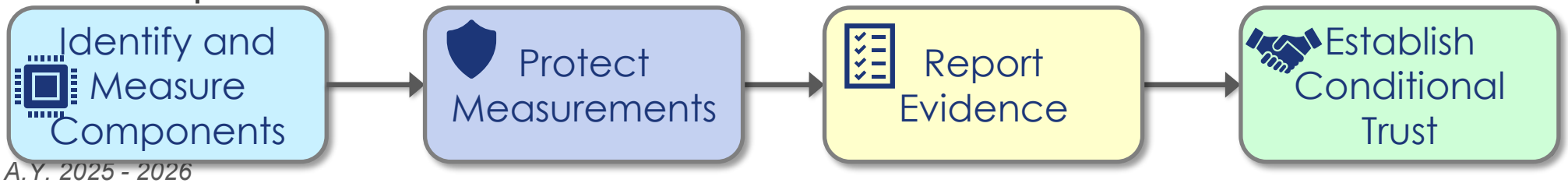
- anchor trust in hardware
- protect cryptographic keys and integrity measurements
- support measurement of the boot process
- report platform state to external verifiers
- provide a common, interoperable standard

The TPM was designed to make platform state verifiable, even when high-privileged software may be compromised.

Trusted Computing goal

Trusted Computing addresses platform-state trust:

1. **identify** the platform components relevant for the trust decision
   - firmware, bootloader, kernel, drivers, configuration, and security policy
2. **measure** security-relevant components
   - a measurement is normally a cryptographic hash of a component or configuration value
   - if the component changes (i.e., its identity changes), the hash changes
3. **protect** the measurements from software tampering
4. **report** integrity evidence to a verifier
   - Remote Attestation
5. **compare** the evidence with expected values or policies and decide whether the platform can be trusted



From HSMs to TPMs: what changes?

Hardware Security Module (HSM)	Trusted Platform Module (TPM)
Protects keys and secrets	Protects keys and platform measurements
Performs cryptographic operations	Performs cryptographic operations and platform-state reporting
May be removable or external	**Bound to the platform**
Not necessarily present during boot	**Available from the earliest boot phases**
It does not represent platform identity	**It can represent platform identity**
⇒ Does not typically know platform state	Designed to support platform measurement and attestation

A **TPM** is not just a cryptographic module. It is a **platform trust anchor**.

What is a TPM?

The TPM 2.0 specification defines the Trusted Platform Module as a device that enables trust in computing platforms.

Trusted Platform Module

TRUSTED® COMPUTING GROUP

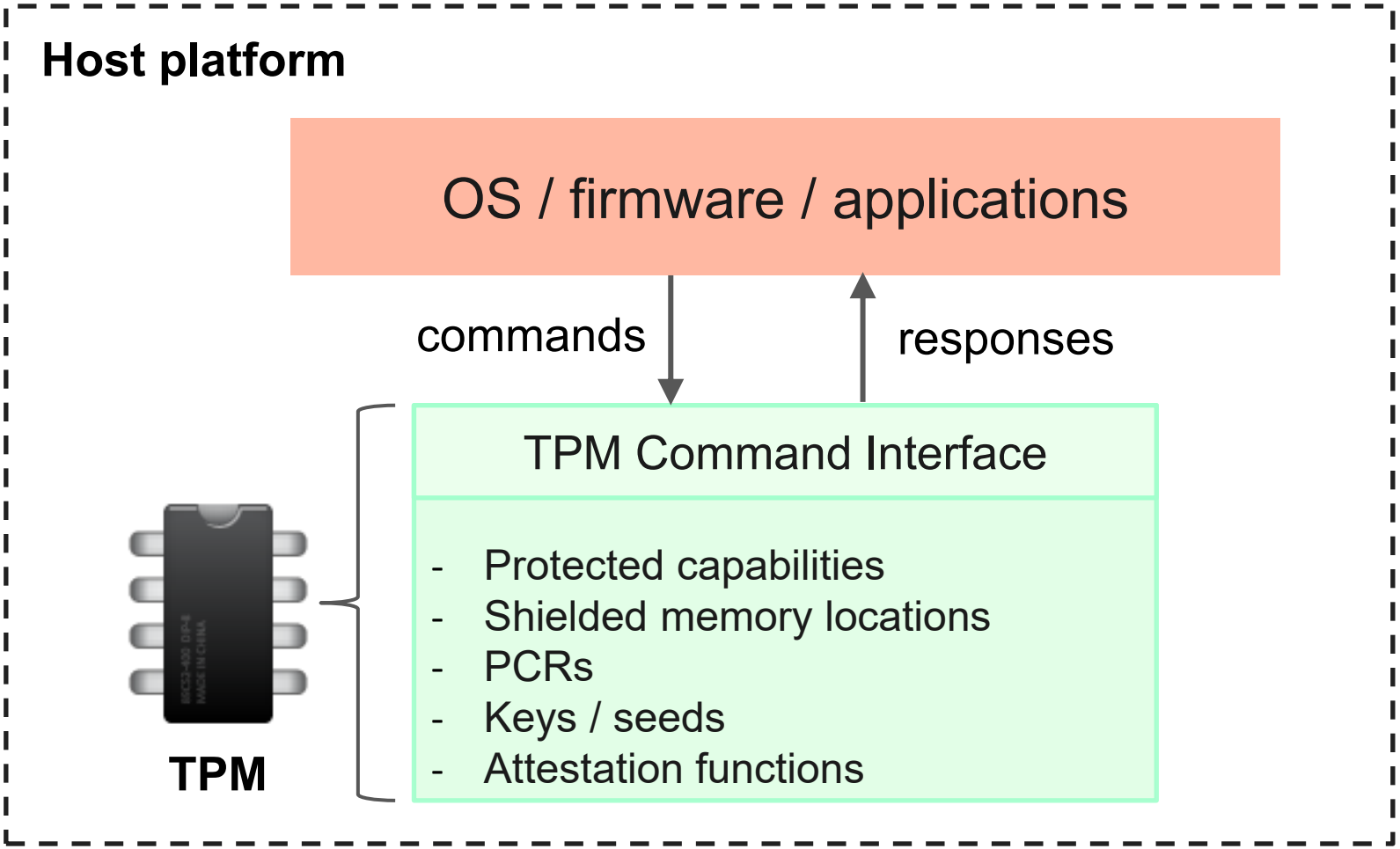
Technical interpretation:

A TPM is a standardized security component that provides protected functions for:

- storing and operating on protected security state
- protecting keys, seeds, authorization values, and measurements
- maintaining **Platform Configuration Registers (PCRs)**
- executing TPM commands through a command/response interface
- **producing authenticated evidence about selected TPM state**

A TPM is not a general-purpose co-processor. We cannot install arbitrary applications inside a TPM . It is a controlled security component that exposes **protected capabilities** to a host platform.

TPM


Host platform
OS / firmware / applications
commands
responses
TPM Command Interface
- Protected capabilities
- Shielded memory locations
- PCRs
- Keys / seeds
- Attestation functions
TPM

TPM implementation types

TRUST ELEMENT	SECURITY LEVEL	SECURITY FEATURES	TYPICAL APPLICATION
DISCRETE TPM	HIGHEST	TAMPER RESISTANT HARDWARE	CRITICAL SYSTEMS
INTEGRATED TPM	HIGHER	HARDWARE	GATEWAYS
FIRMWARE TPM	HIGH	TEE	ENTERTAINMENT SYSTEMS
SOFTWARE TPM	NA	NA	TESTING & PROTOTYPING
VIRTUAL TPM	HIGH	HYPERVISOR	CLOUD ENVIRONMENT

From TPM 1.2 to TPM 2.0

TPM 2.0 introduced important architectural improvements:

- **algorithm agility:** support for multiple cryptographic algorithms
- **separate key hierarchies:** platform, storage, endorsement, null
- **enhanced authorization:** passwords, HMAC sessions, and policies
- **cryptographic object names:** stronger binding between objects and identity

TPM 1.2	TPM 2.0
tightly linked to RSA and SHA-1	algorithm agility
one main storage hierarchy	multiple key hierarchies (Platform hierarchy, Storage hierarchy, Endorsement hierarchy, and Null hierarchy)
less flexible authorization	enhanced authorization mechanism
resources identified mainly by handles	resources identified by cryptographic names
mostly PC-oriented design	broader platform flexibility

What is a Trusted Platform?

*In the TCG sense, **a platform is trusted** if it **behaves as expected for a specific purpose**.*

Important clarification: **Trusted ≠ Secure**

- **Secure**: enforces a defined security policy under a defined threat model.
- **Trusted**: behaves in a predictable or expected way.

Examples:

- a bank is trusted to provide financial services
- a thief may also be “trusted” to steal

⟹ **Predictable behavior is not necessarily secure or desirable behavior**.

Trusted ≠ Secure

SECURE SYSTEM

- enforces a security policy under a threat model
- Question: “Is the policy enforced?”

TRUSTED PLATFORM

- behaves as expected for a specific purpose
- Question: “Is this the platform state I expected?”

To decide whether a platform is trusted, we need evidence about:

- hardware components
- firmware and boot code
- operating system and drivers
- security configuration

Roots of Trust in a Trusted Platform

A Root of Trust is a component that must always behave in the expected manner because its misbehavior cannot be detected.

- A RoT provides a security function the system relies on, whose correct behaviour is assumed by design, certification, and secure integration
- However, trust is not blind: a RoT can be certified / validated according to Common Criteria and/or FIPS frameworks
  - e.g., CC EAL4+, FIPS 140 Level 1/Level 2
- According to the TCG model, a **Trusted Platform** must provide a minimum set of Roots of Trust:

1. **Root of Trust for Measurement (RTM)**
2. **Root of Trust for Storage (RTS)**
3. **Root of Trust for Reporting (RTR)**

Roots of Trust in a Trusted Platform

Root of Trust	Definition	In a TPM-based platform
RTM	Component that measures platform components or security-relevant events and sends the resulting measurement data to protected storage.	Usually firmware / boot code, starting from the Core Root of Trust for Measurement (CRTM).
RTS	Component that protects security-relevant state in shielded locations, such as measurements, keys, seeds, and authorization values.	Implemented by the TPM through shielded locations, including PCRs and protected objects.
RTR	Component that reports protected state in an authenticated way, so that another entity can verify platform state evidence.	Implemented by the TPM through attestation functions, such as quotes over PCR values.

TPM, RoTs, and the TCB: what the TPM is not

What the TPM implements

In a TPM-based platform, the TPM implements:

- **RTS**: protects keys, seeds, PCRs, and internal TPM state
- **RTR**: reports selected TPM state through signed attestations (**TPM quote** operation)

What is outside the TPM

- **RTM**: platform code that calculates digests over configuration data and program code, and sends them to the RTS.

CRTM, and Chain of Trust for Measurement

- **Core Root of Trust for Measurement (CRTM):**
  the first immutable component that takes control of the platform after reset. Its role is to measure the first mutable code and store the measurement in the TPM before transferring control to the measured component.
- **Chain of Trust for Measurement (CoTM):**
  a transitive measurement chain in which each component measures the next component before passing control to it:
  - the CRTM measures the next component that takes control of the platform at boot
  - if that component is considered trusted (i.e., its measurement is as expected), we can trust the measurements that it will take on the following components
  - and so on …
  - example: CRTM measures firmware → firmware measures bootloader → bootloader measures kernel → kernel may measure later components.

Trusted Computing Base (TCB)

The collection of system resources responsible for enforcing and maintaining the system security policy.

- The TCB may include hardware, firmware, bootloaders, kernel, hypervisor, drivers, and security services.
- These components enforce the security policy: isolate processes, manage memory, control access to files and devices, mediate system calls and hardware access

The TPM is not the TCB of the host system; rather, it is a component used to help determine whether the host TCB has been instantiated correctly or has been compromised.

- In some uses, the TPM may help prevent the system from starting if the TCB cannot be properly instantiated.

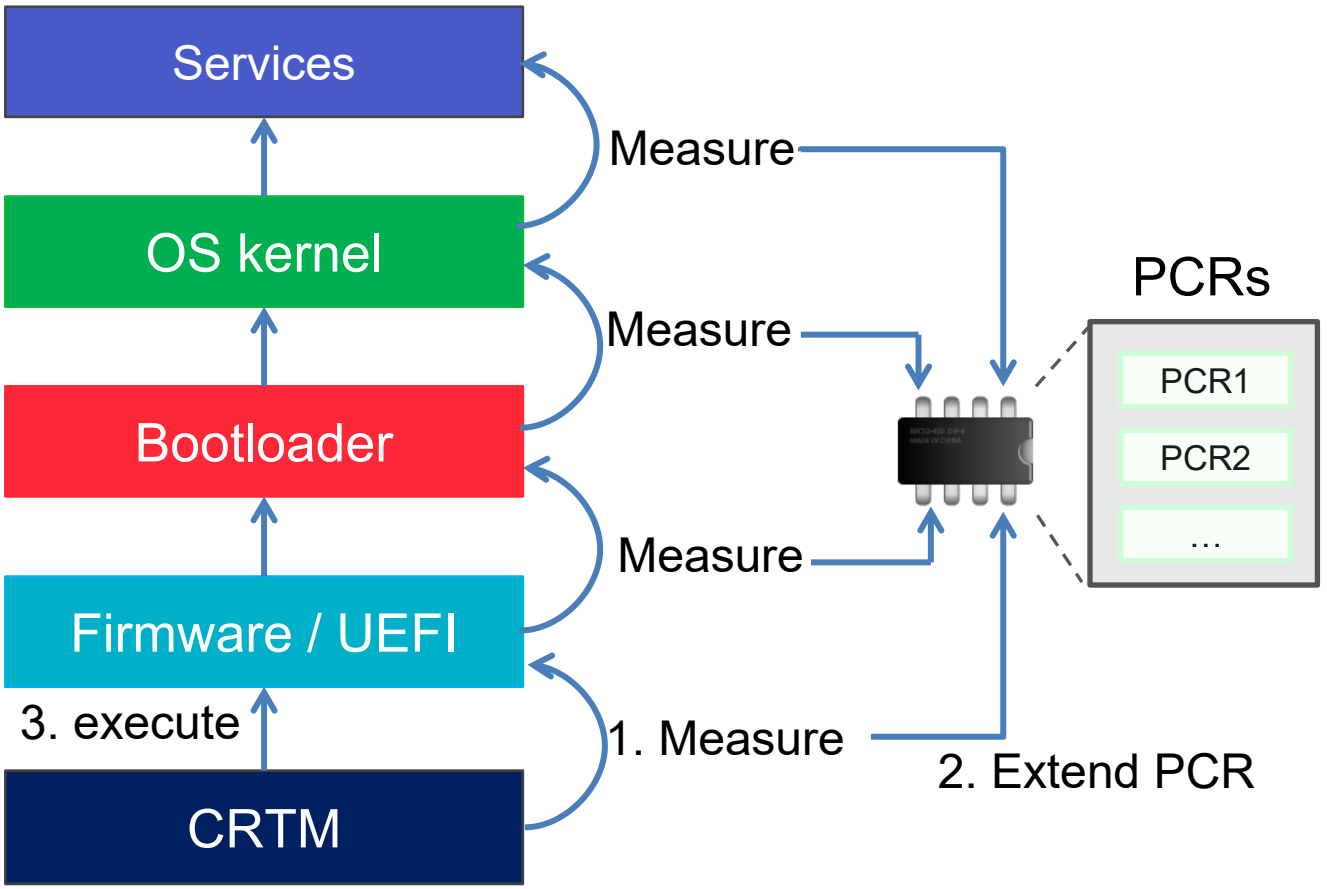
Measured Boot

Measured Boot *is the process by which security-relevant components loaded during platform start-up are measured and their measurements are stored in TPM PCRs.*

- During boot, the platform records evidence about:
  - firmware and platform initialization code
  - boot managers and bootloaders
  - boot configuration and security policies
  - operating system kernel and early drivers
- A measurement is normally a cryptographic hash of the component, or of relevant configuration data
- that measurement is then sent to the TPM and accumulated inside a PCR
- Measured Boot records what was loaded. **It does not prevent a modified component from executing**.

The goal is to make the boot state verifiable after the platform has started.

Measured Boot: measure before execute



Measured Boot follows a **measure-before-execute** principle:

1. measure the next component
2. extend the measurement into a PCR
3. transfer control to the measured component

Each subsequent component repeats the process.

Result: final PCR values depend on the sequence of measured boot components.

PCR Extend Operation: accumulating measurement history

- A PCR is not overwritten with the latest measurement
  - if that were possible, a compromised platform could erase bad measurements and replace them with good ones
- Instead, the TPM updates it through the **extend** operation:

$$PCR_{new} = Hash(PCR_{old} \parallel measurement)$$

Where:

- PCR_{old}: current PCR value
- $measurement$: hash of a measured component or event
- $Hash$: cryptographic hash function
- $\parallel$: concatenation

Key consequence: The final PCR value depends on:

1. all measurements extended into it;
2. their exact order.

PCRs behave like a tamper-evident summary of measurement history, not like normal registers.

Typical PCR usage during PC boot

- The TPM PCRs are divided into banks; a bank represents the set of PCRs all extended with the same hash algorithm
  - For example: a TPM can have the SHA-1 bank, SHA-256 bank, and so on
- Each bank typically contains 24 PCRs, indexed from PCR[0] to PCR[23]
- Different PCRs are normally used to store measurements of different parts of platform state
- Typical PC Client usage of TPM PCRs:
- **PCR 0–7:** firmware, platform configuration, boot manager, secure boot policy
- **PCR 8–15:** static operating-system boot measurements
- **PCR 16:** debug
- **PCR 17–23:** application support
- **Important:** a PCR value is meaningful only if we know what was extended into it.

Typical PCR usage during PC boot

PCR index	Typical usage
0	CRTM, BIOS/UEFI platform code, host platform extensions
1	Host platform configuration
2	UEFI driver and application code
3	UEFI driver configuration and data
4	UEFI Boot Manager code and boot attempts
5	Boot Manager configuration and partition table data
6	Host platform manufacturer-specific data
7	Secure Boot policy
8 – 15	Static OS use, bootloader/kernel-related measurements
16	Debug
23	Application support

PCR meaning is defined by platform profiles and system configuration.

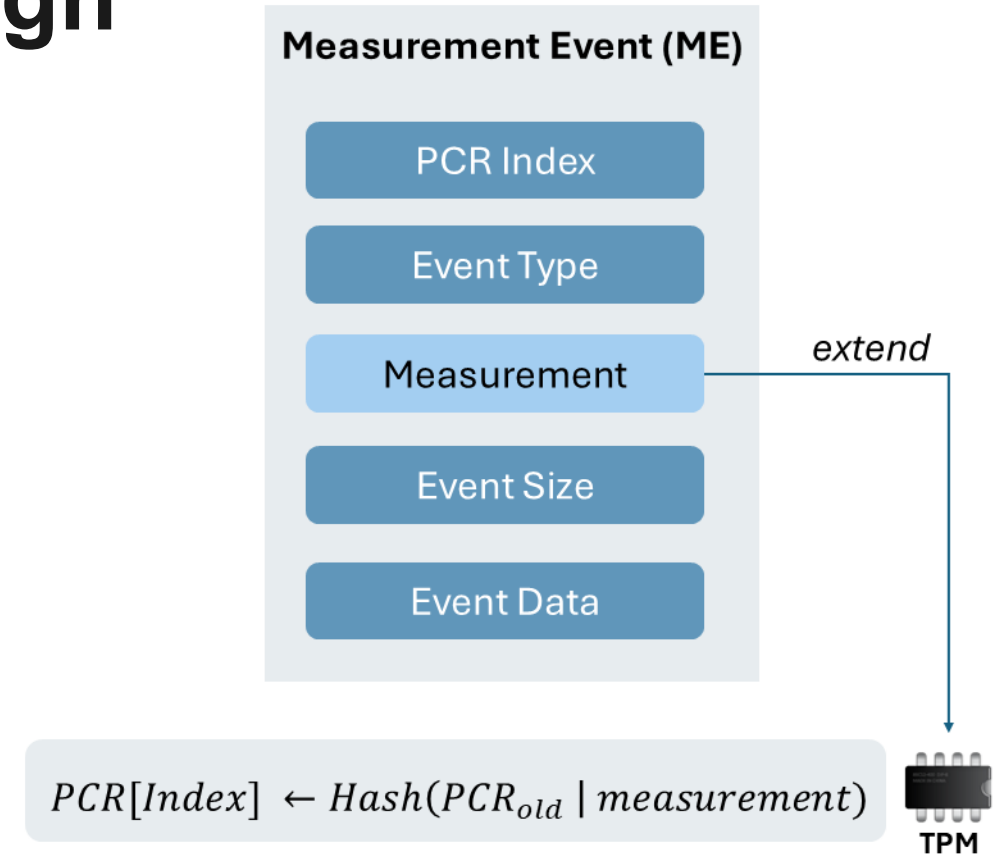
Measurement Event Log: why PCRs are not enough

PCR[4] = `a3f95dd...07c21`

Question: What exactly caused this value?

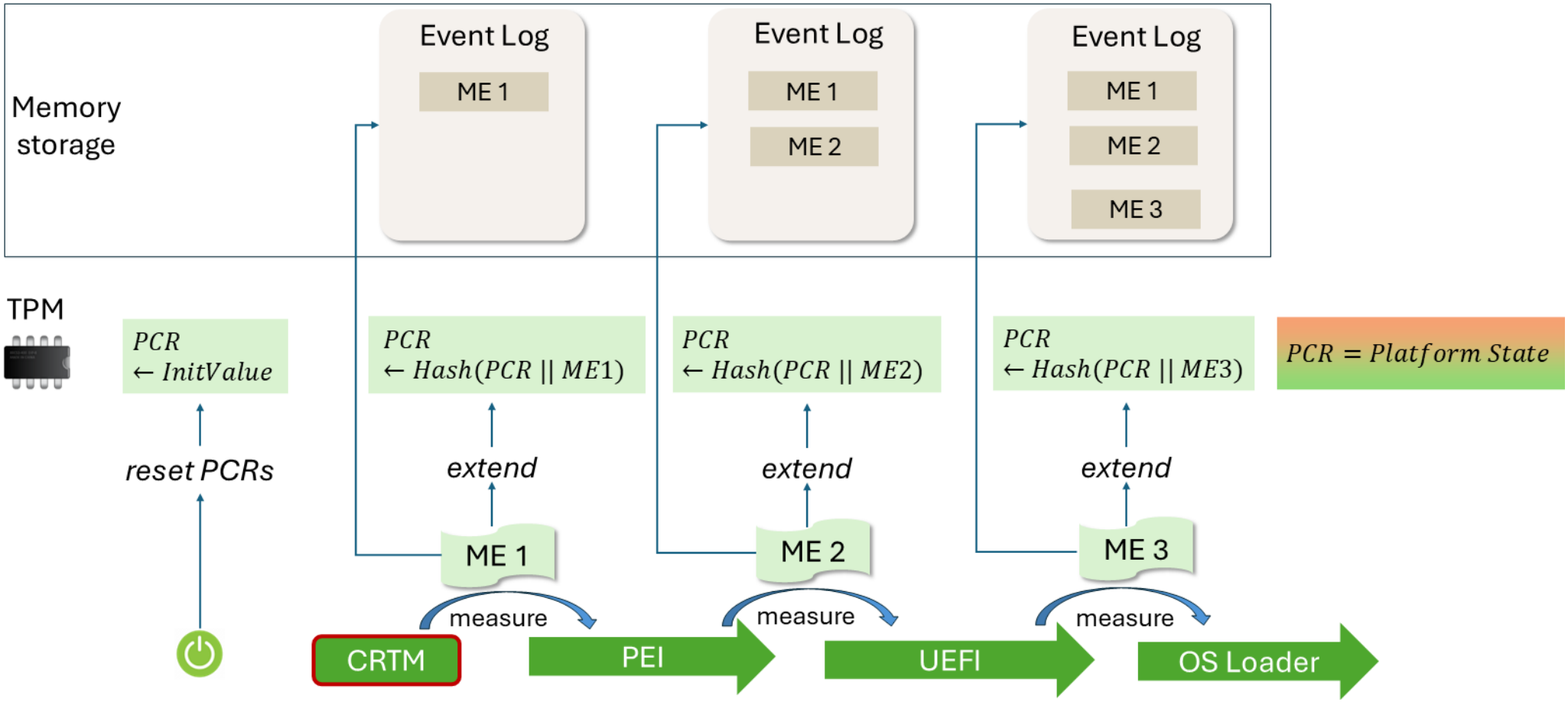
- If only one measurement (or a small predetermined set of measurements) is extended in a PCR, its value is descriptive, as we can compare the final value with the expected value.
- If the PCR value results from a sequence of extensions depending on highly variable configuration parameters, or whose order is not deterministic, the final value cannot be used alone to understand if all components are as expected.
- The **Integrity Measurement Log**, or **Event Log**, records each **Measurement Event (ME)** occurred in the system: which PCR was extended, what type of event occurred, which digest was extended, what data describes the event.

Measurement Event Log: why PCRs are not enough



TPM PCR: protected aggregate of measurement events
Event Log: detailed explanation of the measurement events occurred in the system

Event Log recording during Measured Boot



Verifying an Event Log

The event log is outside the TPM, so its integrity must be checked against the protected PCR value. This is needed to detect whether a malicious component was trying to hide its presence in the system.

Verification procedure:

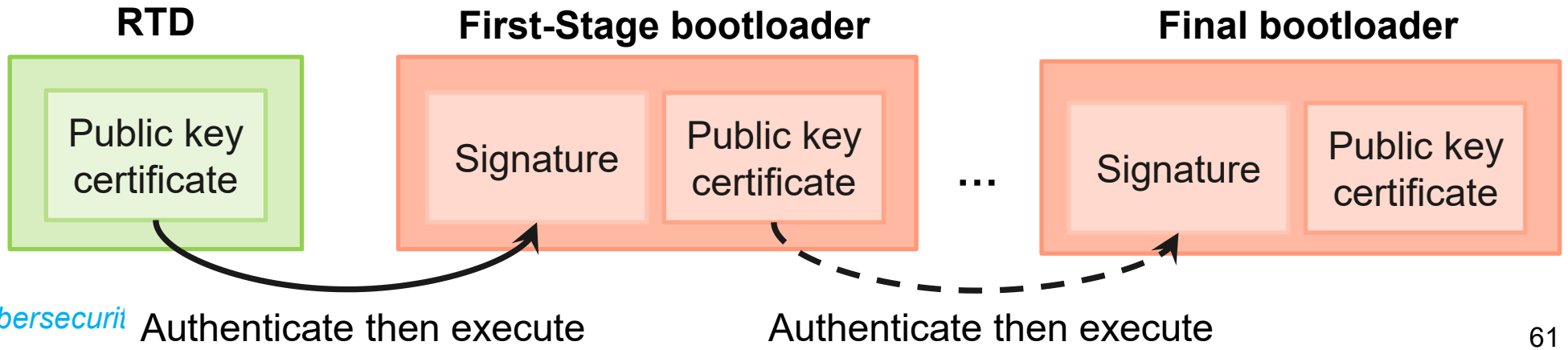
1. start from the PCR reset value
2. read the first event-log entry
3. recompute its measurement
4. recompute the PCR extend operation
5. repeat for all log entries
6. compare the recomputed PCR with the TPM-reported PCR:
   - **if the two values match**, the log is consistent with the TPM-protected PCR state ⇒ it has not been tampered with, it's authentic ⇒ each individual entry can be inspected in order to decide whether all components are acceptable
   - **if the two values do not match**, something is wrong, the event log may have been modified, truncated, reordered, or it may simply not correspond to the current PCR state

Secure Boot

Secure Boot *verifies that a boot component is authorized before allowing it to execute.*

For checking authenticity and integrity, Secure Boot uses:

- trusted public keys or certificates
- digital signatures or trusted hashes
- boot policy
- a Root of Trust for Detection / Verification (RTD / RTV)

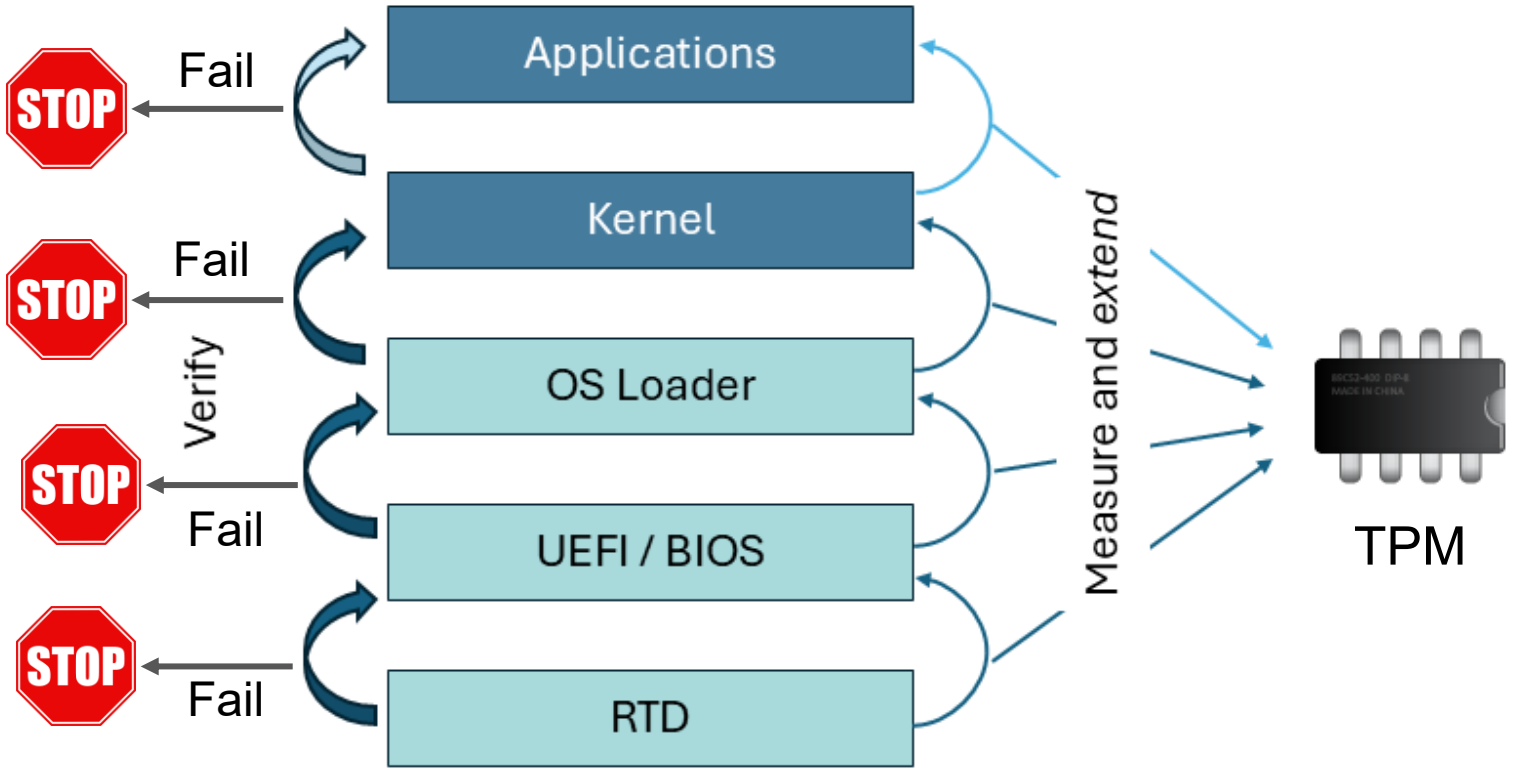


Secure Boot: verify before execute

- Secure Boot follows a **verify-before-execute** principle: before a boot component is executed, the current stage checks whether the next component is authorized.
- Its objective is to **prevent unauthorized components from running.**
- Verification is usually based on digital signatures, trusted public keys, certificates, or trusted hashes
  - **if verification succeeds**, the next component is executed.
  - **if verification fails**, the boot process stops, or the platform enters a recovery (immediate protection)
- The verify-before-execute procedure starts in the RTD, which possibly delegates next stages verification to a Chain of Trust for Detection (CTD).
- Secure Boot typically protects firmware and bootloader, optionally kernel and drivers.

Secure Boot vs Measured Boot

Secure Boot VS Measured Boot

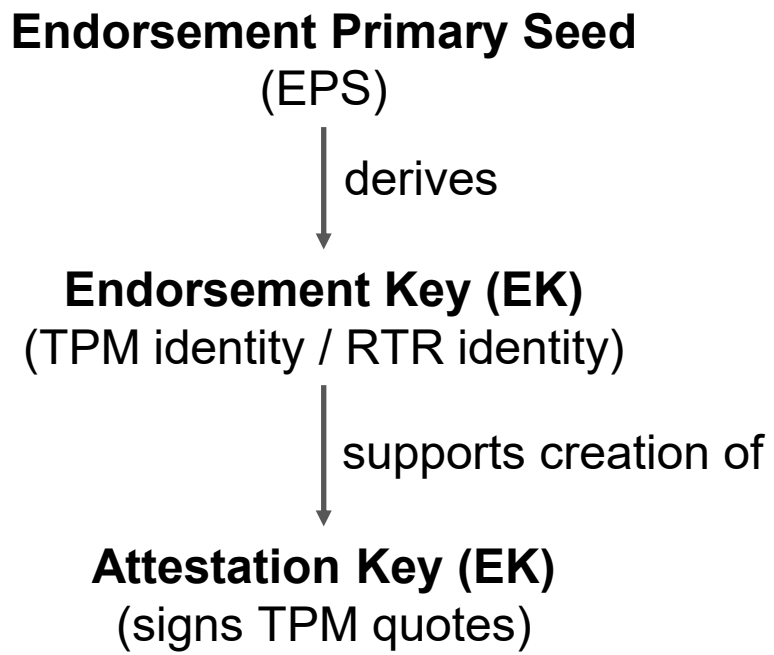


Measured Boot records *evidence* (*detect later*)
Secure Boot enforces a boot decision (*block now*)

TPM Endorsement Key and Attestation Key

- **Endorsement Key (EK)**:
- represents the TPM identity
- **identity key** on which the TPM RTR is founded
- derived from the **Endorsement Primary Seed** (EPS)
- typically associated with **EK Certificate**, issued by the TPM manufacturer
- typically never used for signing operations (it is normally used as a decryption key)
  - this protect privacy, when the device on which the TPM is attached belongs to a human user
  - this protects the private part of the EK
- **Attestation Key (AK)**:
- TPM-resident key used to sign attestation evidence
- The TPM command to sign selected PCR values (with freshness data) is called **TPM quote**
- an AK can be certified through the EK
- avoids exposing the EK in every attestation

Key hierarchy diagram



Sealing: binding secrets to platform state

Sealing means protecting data with a TPM policy.

A sealed object can be released only if:

- it is accessed on a platform having the correct TPM (**TPM binding** concept), and
- the current platform state (represented by PCR values) satisfies the required PCR policy (i.e., PCRs must match a given value) (**TPM sealing** concept)

Example:

Release the disk encryption key only if firmware, bootloader, and Secure Boot policy match the approved state.

Typical use cases:

Disk encryption keys, VPN credentials, device identity keys, application secrets

Warning: Sealing is powerful, but PCR policies must be designed carefully: firmware/bootloader/kernel updates may change PCR values involved in the sealing policy ⇒ legitimate updates may prevent unsealing.

TPM Software Stack: how applications use the TPM

Applications usually do not interact with the TPM through raw commands, it would be too difficult and error-prone.

- TPM commands have specific binary formats
- parameters must be marshaled into command byte streams and unmarshaled from response byte streams
- many commands require sessions, authorization values, HMACs, encryption of command parameters, and management of TPM objects
- also, TPM internal memory is limited, so objects may need to be loaded, evicted, and restored.

The **TPM Software Stack (TSS)**, provides layered software APIs for: easier TPM programming; session and authorization handling; command encoding and response decoding; TPM resource management; safe access by multiple applications; support for local, virtual, simulator, and remote TPMs.

TSS 2.0: how applications use the TPM

TSS 2.0 is a software bridge between applications and the TPM.

From application to TPM:

- **Application / tools –** request TPM services: keys, sealing, quotes, attestation.
- **Feature API (FAPI)** – easiest interface: high-level functions for common TPM use cases.
- **Enhanced System API (ESAPI)**: full TPM access, with sessions, HMACs and encryption handled by the library.
- **System API (SAPI)** + **Marshaling/Unmarshaling API (MUAPI)**: low-level command mapping and marshaling/unmarshaling of TPM data.
- **TPM Command Transmission Interface (TCTI)** – transport layer: sends TPM commands to a local TPM chip, TPM simulator, vTPM or daemon.
- **TPM Access Broker (TAB)** / **Resource Manager (RM) + OS driver**: serialize access and manage scarce TPM resources.

TSS 2.0 overview

NOTE: This is the logical TSS 2.0 model. In modern Linux, the Resource Manager is often implemented in the kernel through /dev/tpmrm0; tpm2-abrmd is still available as an optional user-space TPM Access Broker / Resource Manager.